

Yes. Use the **FINAL ZIP only** from now on.

Download CoveragePrep Simple — FINAL

First-time setup

1. Download the ZIP.
2. Right-click it → **Extract All**.
3. You should get:

```
CoveragePrep_Simple_FINAL/
```

Inside it you'll see:

```
main.py
SETUP_AND_RUN_WINDOWS.bat
RUN_WINDOWS.bat
requirements.txt

coverageprep/
input/
output/
config/
sample_data/
tests/
```

Do **not** run the program from inside the ZIP itself. Extract it first.

Easiest way to run

For the **first time only**, double-click:

```
SETUP_AND_RUN_WINDOWS.bat
```

That script should automatically:

```
Create .venv
↓
Install Python packages
↓
Start CoveragePrep
```

You may see a black Command Prompt window while it installs packages. Leave it open.

When setup finishes, you should see the menu:

```
=====
CoveragePrep Simple
Intelligent Client Shipment Preparation
=====

1. Scan a shipment file
2. Prepare a shipment file
3. Reuse a saved configuration
4. Exit

Select option:
```

After the first setup, you don't need to reinstall everything each time.

Next time simply double-click:

```
RUN_WINDOWS.bat
```

I recommend running it from VS Code

Since you're familiar with VS Code, this is the best way to understand what is happening.

Open VS Code → **File** → **Open Folder** → select:

```
CoveragePrep_Simple_FINAL
```

Then:

Terminal → **New Terminal**

Your terminal should be inside something like:

```
C:\Users\...\CoveragePrep_Simple_FINAL>
```

First verify Python:

```
bat

python --version
```

You should see something such as:

```
Python 3.11.x
```

Then create the environment if you haven't already:

```
bat  
  
python -m venv .venv
```

Activate it:

```
bat  
  
.venv\Scripts\activate
```

Your terminal should now begin with:

```
(.venv)
```

Install the required packages:

```
bat  
  
python -m pip install -r requirements.txt
```

Then run:

```
bat  
  
python main.py
```

First test — don't use a real client file yet

When the menu appears:

```
1. Scan a shipment file  
2. Prepare a shipment file  
3. Reuse a saved configuration  
4. Exit
```

Choose:

```
2
```

For your first test, use the included sample file.

When asked for the file, select or enter:

```
sample_data\sample_multisheet_shipment.xlsx
```

This lets you learn the program without touching confidential data.

What the four menu choices mean

1. Scan a shipment file

Use this when you want CoveragePrep to **inspect** a file but not transform it.

It checks things such as:

```
all sheets
sheet sizes
likely header rows
multi-row headers
period structures
possible issues
```

Good for an unfamiliar shipment.

2. Prepare a shipment file

This is the main option.

It walks you through:

```
Select file
↓
Select sheets
↓
Confirm headers
↓
Detect periods
↓
Map fields
↓
Confirm FACT/value
↓
Transform
↓
Validate
↓
Create output
```

3. Reuse a saved configuration

Use this later when you receive another shipment with the **same structure**.

For example:

```
January file
↓
Configure manually once
↓
save config\Client_Config.json

February file
↓
reuse configuration
```

The program still rescans the February file first and warns you if the structure changed.

4. Exit

Closes the program.

How to use an actual client shipment

For real client files, you have two safe choices.

You can place the file inside:

```
input/
```

For example:

```
CoveragePrep_Simple_FINAL/
|
├─ input/
|   └─ Client_Shipment.xlsx
```

Then run:

```
bat

python main.py
```

and select it.

Or leave the client file wherever it already exists on your company laptop and enter its full path, for example:

```
C:\Users\...\Documents\ClientShipment.xlsx
```

I actually prefer the second method for confidential data because you don't need to copy the shipment into the project folder.

Where does the result go?

Successful output is saved in:

```
output/
```

For example:

```
output\Client_Shipment_Prepared.xlsx
```

The workbook contains sheets such as:

```
Prepared_Data  
Source_Copy  
Field_Mapping  
Period_Map  
Validation  
Issues  
Recommendations  
Audit_Log
```

If validation does not pass, you should instead get:

```
Client_Shipment_REVIEW_REQUIRED.xlsx
```

That means you should review the detected issue before treating the prepared data as final.

What is the `config` folder?

After successfully preparing a shipment, you can save its transformation configuration there:

```
config\Client_Config.json
```

That remembers things such as:

```
selected sheets  
header rows  
field roles  
fact definition  
period rules  
period ordering  
negative-value rule  
duplicate rule
```

It does **not** need to contain the actual shipment values.

What should you never put on GitHub?

For this project, never commit actual client files.

Do not upload:

```
input\Client_Shipment.xlsx  
output\Client_Shipment_Prepared.xlsx  
config files containing confidential details  
API tokens  
passwords
```

Keep real client processing local in the approved company environment.

Your normal daily workflow

After the one-time setup, it's basically:

```
Open CoveragePrep_Simple_FINAL in VS Code  
↓  
Open Terminal  
↓  
.venv\Scripts\activate  
↓  
python main.py  
↓  
Choose 1, 2, or 3
```

So the only commands you'll normally need are:

bat

```
.venv\Scripts\activate  
python main.py
```

If you want the simplest possible method, just double-click:

```
RUN_WINDOWS.bat
```

after the first setup.

For your first run, use the **sample multi-sheet workbook**, and if you reach a prompt you're unsure about, send me a screenshot of that exact terminal screen and I'll tell you what to select.